

404 CTF - Édition 2026

Thème : Les grandes figures scientifiques françaises !

Writeup : Curieux SMS

Catégorie : Analyse Forensique

Auteur : syco

Date de résolution : 25/05/2026

1. Énoncé

Challenge

215 résolutions

✕

Curieux SMS

100

easy

syco

Plusieurs chercheurs sont suspectés d'avoir divulgué des informations classifiées. Leurs téléphones ont été saisis et une image forensique de la base de données SMS a été extraite.

 chall.zip

Flag

Soumettre

2. Solution

Voici le blog qui m'a permis de comprendre un minimum le contexte pour analyser des SMS supprimés :

https://az4n6.blogspot.com/2013/02/finding-and-reverse-engineering-deleted_1865.html

Dans ce blog il préconise l'outil "Cellebrite", mais celui-ci est payant.

J'ai donc cherché une alternative, et j'ai donc utilisé Autopsy qui est une boîte à outil avec plusieurs modules pour le forensique.

Voici la création du projet Autopsy :



New Case Information

Steps

1. Case Information

2. Optional Information

Case Information

Case Name: CurieuxSMS

Base Directory: Browse

Case Type: ☒ Single-User ☐ Multi-User

Case data will be stored in the following directory:
Downloads\chall\CurieuxSMS

< Back

Next >

Finish

Cancel

Help

Add Data Source

Steps

1. Select Host

2. Select Data Source Type

3. Select Data Source

4. Configure Ingest

5. Add Data Source

Select Host

Hosts are used to organize data sources and other data.

☒ Generate new host name based on data source name

☐ Specify new host name

☐ Use existing host

< Back

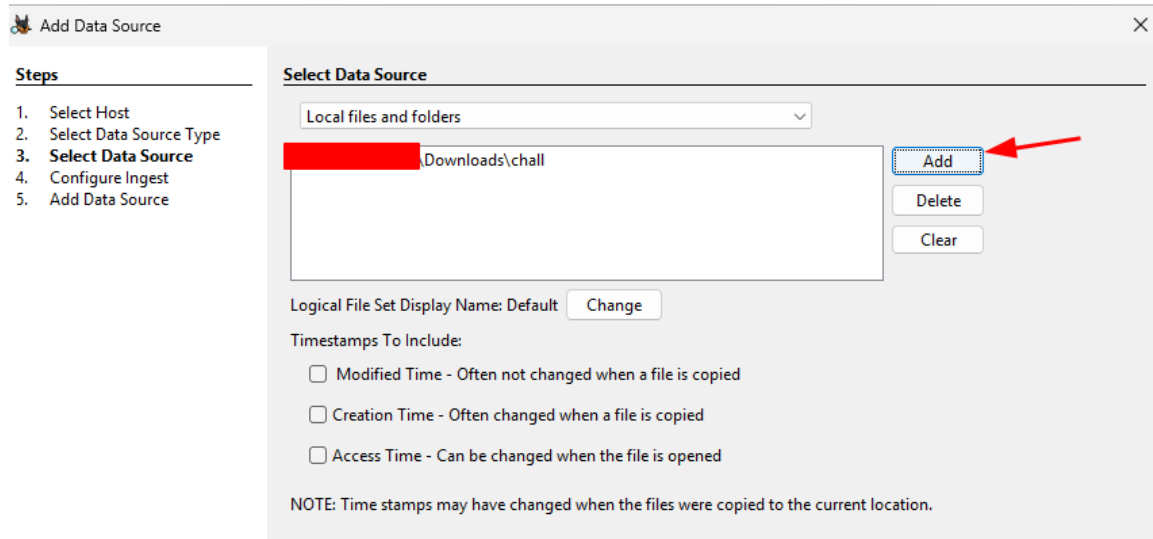
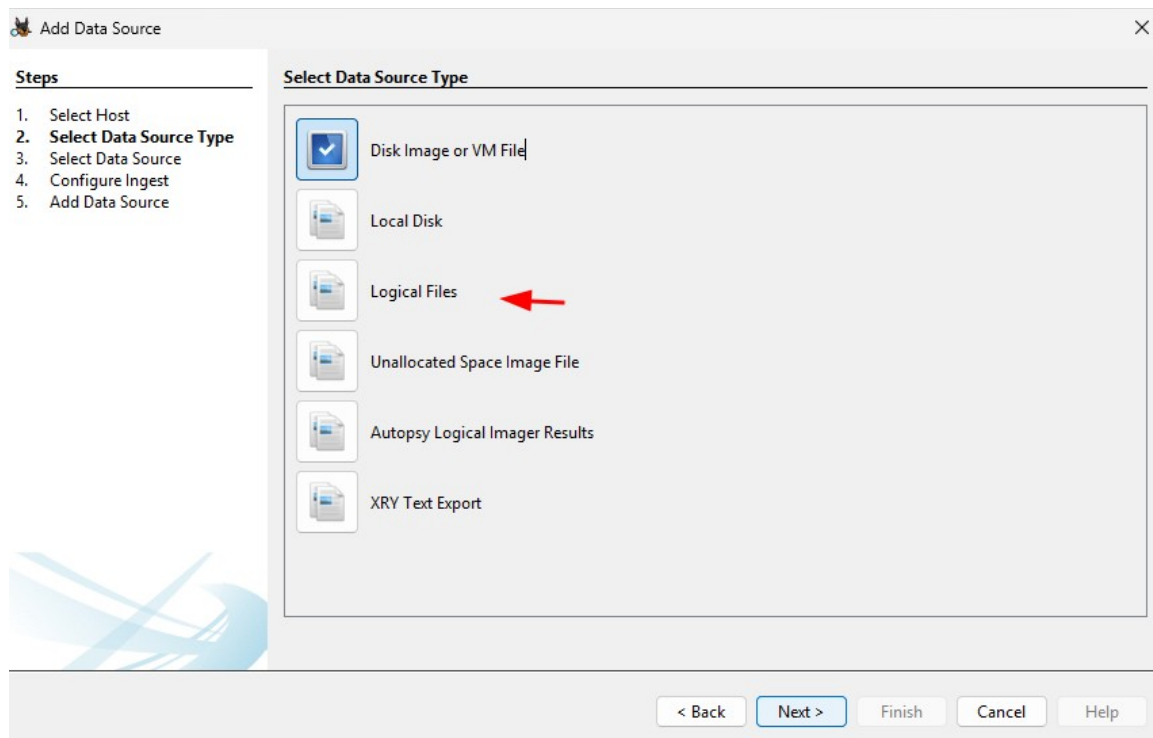
Next >

Finish

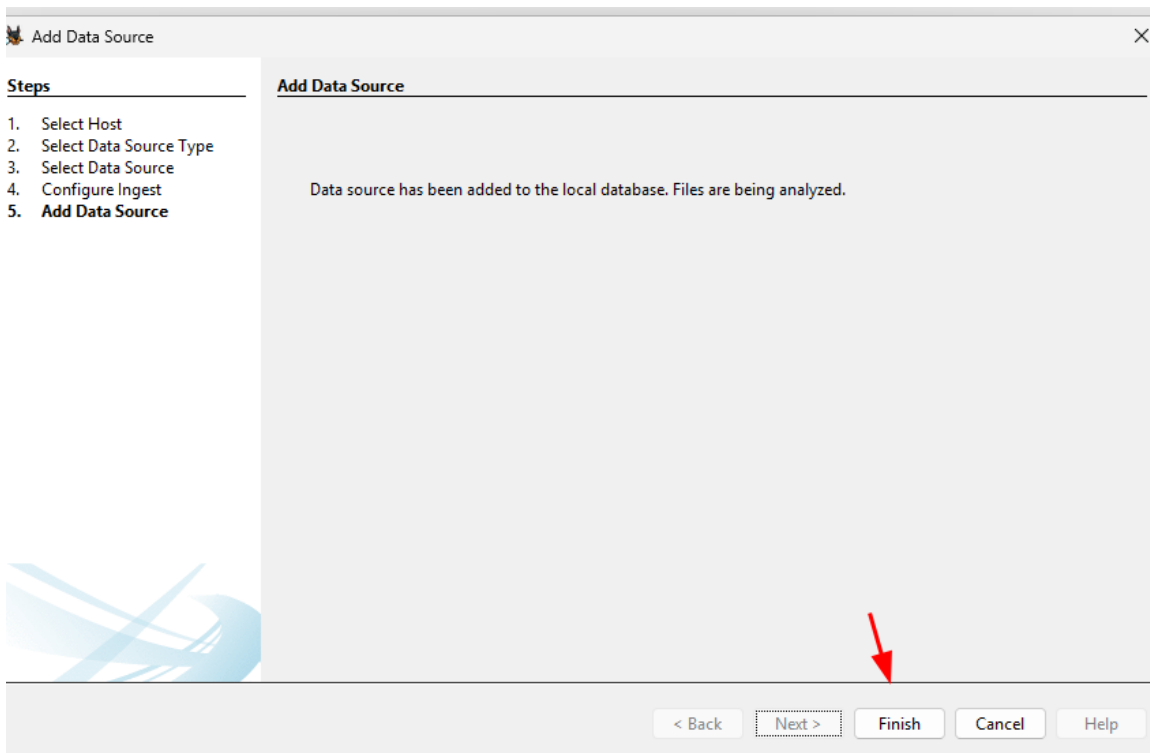
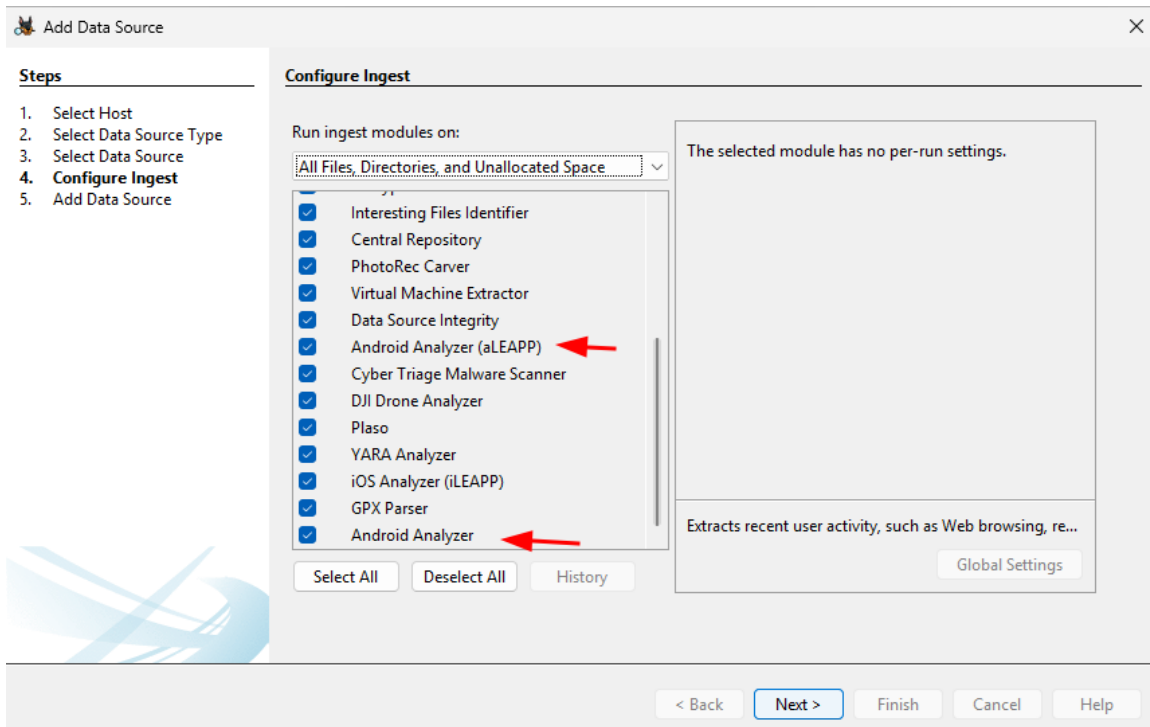
Cancel

Help

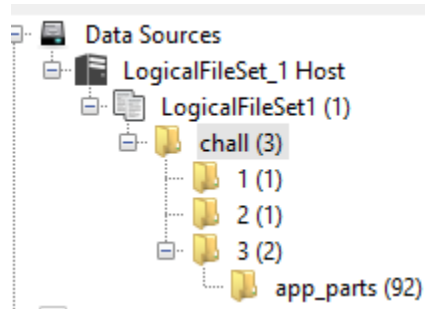
Il faut choisir le type “Logical Files” comme source de données :



Nous avons plusieurs modules au sein de Autopsy, à noter qu'il existe également des modules pour des analyses Android :



Au sein de ce projet, nous avons 3 dossiers, dans chaque dossier nous avons une base de données nommée “mmssms.db”, et dans le troisième dossier nous avons un sous-dossier supplémentaire “app_parts” :



D’après le blog ci-dessus, ces bases de données sont aux formats SQLite, et au sein d’un smartphone Android elles sont localisées dans le chemin :

/Root/data/com.android.providers.telephony/databases

Ces bases de données conservent les données supprimées, nous allons donc tout analyser pour trouver des indices.

Analyse de la première base de données, dans la table “sms” :

body	service_c...	loc
Tu as vu les résultats de l'expérience ?		0
C'est une découverte majeure		0
Il faut garder ça secret		0
Les échantillons sont prêts		0
Ne parle à personne de cette experience		0
Le mot de passe pour accéder au dossier est 404CTF{m4r13_		0

Nous avons une première partie d’un flag 404CTF !

Vérification de la deuxième base de données dans la table “conversations”, qui permet également de récupérer un bout de flag :

Hex	Text	Application	File Metadata	OS Account	Data Artifacts	Analysis Results	Context	Annotations	Other Occurrences
Table		conversations	1 entries		Page 1 of 1		Export to CSV		
_id	sms_thre...	name	latest_m...	snippet_...	subject_t...	preview_...	preview_...	show_dr...	
	1	0698725361	4	cur13_				0	

Ensuite, dans le dossier “app_parts” avec le fichier “62”, nous avons également un flag :

LogicalFileSet1 (1)

- chall (3)
 - 1 (1)
 - 2 (1)
 - 3 (2)
 - app_parts (92)

sws

- Types
- eted Files
- Size
- rtifacts
- is Results
- counts

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)	Known	Location
56				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	333	Allocated	Allocated	unknown	/LogicalFile!
59				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	1164	Allocated	Allocated	unknown	/LogicalFile!
6				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	3898	Allocated	Allocated	unknown	/LogicalFile!
60				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	2399	Allocated	Allocated	unknown	/LogicalFile!
61				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	333	Allocated	Allocated	unknown	/LogicalFile!
62				0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	0000-00-00 00:00:00	2575	Allocated	Allocated	unknown	/LogicalFile!

HexTextApplicationFile MetadataOS AccountData ArtifactsAnalysis ResultsContextAnnotationsOther Occurrences

0°433%Reset

r4d1um_

Et enfin, nous pouvons trouver le derniers élément du flag dans la troisième base de données au sein de la table "threads" :

Hex	Text	Application	File Metadata	OS Account	Data Artifacts	Analysis Results	Context	Annotations	Other Occurrences
Table: threads 1 entries Page 1 of 1 Export to CSV									
_id	date	message...	recipient_ids	snippet	snippet_cs	read	archived	type	error
1	1778951853000	1	+33698765432	1898}	106	1	0	0	0

A ce moment là, nous avons tout les éléments pour forger le flag !



4. Obtention du Flag

Une fois toutes ces informations rassemblées, cela donne :

404CTF{m4r13_cur13_r4d1um_1898}